

MODUL 7

Nama: Ryan Akeyla Novianto Widodo

Kelas: 12 IF 07

NIM : 103112400081

DASAR TEORI

Dasar Teori C++

Diambil dari internet dan dirangkum ulang oleh saya dan dibantu dengan AI.

1. Inti Program

- Syntax: Mirip bahasa Inggris, tapi huruf besar dan kecil beda arti. Setiap perintah biasanya diakhiri dengan titik koma (;).

- Komentar: Catatan di kode yang tidak dijalankan.

- Satu baris: // Ini komentar

- Banyak baris: /* Ini komentar yang panjang */

- Header: File yang berisi daftar perintah dan fungsi yang bisa dipakai. Contoh:

cpp

```
#include <iostream> // Untuk perintah input/output (cout, cin)
```

- main() : Fungsi utama, tempat program mulai berjalan.

cpp

```
int main() {
```

```
    // Kode program di sini
```

```
    return 0; // Menandakan program selesai dengan benar
```

```
}
```

- Namespace: Seperti "area" untuk nama-nama perintah. std adalah namespace standar C++.

cpp

```
using namespace std; // Supaya tidak perlu tulis std::cout, std::cin, dll.
```

- Input/Output:

- cout : Menampilkan sesuatu di layar.

- cin : Membaca input dari keyboard.

cpp

```
cout << "Halo!" << endl; // Menampilkan "Halo!"
```

```
int umur;
```

```
cin >> umur; // Membaca angka dari keyboard dan disimpan di variabel "umur"
```

2. Jenis Data

- Integer (Bilangan Bulat):

- int : Bilangan bulat (contoh: -10, 0, 25).

- short : Bilangan bulat pendek.

- long : Bilangan bulat panjang.

- Floating-Point (Bilangan Desimal):

- float : Bilangan desimal (contoh: 3.14).

- double : Bilangan desimal dengan ketelitian lebih tinggi.

- Character (Karakter):

- char : Satu huruf, angka, atau simbol (contoh: 'a', 'Z', '5').

- Boolean (Logika):

- bool : Nilai benar (true) atau salah (false).

- void : "Kosong". Dipakai untuk fungsi yang tidak menghasilkan nilai.

3. Variabel

- Deklarasi: Memberi tahu program nama dan jenis data variabel.

cpp

```
int umur; // Variabel bernama "umur" untuk menyimpan bilangan bulat
```

```
double harga; // Variabel bernama "harga" untuk menyimpan bilangan desimal
```

- Inisialisasi: Memberi nilai awal ke variabel.

cpp

```
int umur = 20; // "umur" diisi dengan nilai 20
```

```
double harga = 99.50; // "harga" diisi dengan nilai 99.50
```

4. Operator

- Aritmatika: + (tambah), - (kurang), * (kali), / (bagi), % (sisa bagi).
- Penugasan: = (mengisi nilai ke variabel).
- Perbandingan: == (sama dengan), != (tidak sama dengan), > (lebih besar), < (lebih kecil), >= (lebih besar atau sama dengan), <= (lebih kecil atau sama dengan).
- Logika: && (DAN), || (ATAU), ! (TIDAK).
- Increment/Decrement: ++ (tambah 1), -- (kurang 1).

5. Struktur Kontrol

- if : Melakukan sesuatu jika kondisi benar.

cpp

```
if (umur >= 17) {  
    cout << "Anda sudah dewasa." << endl;  
} else {  
    cout << "Anda belum dewasa." << endl;  
}
```

- switch : Memilih salah satu dari beberapa pilihan.

cpp

```
int hari = 3;  
switch (hari) {  
    case 1:  
        cout << "Senin" << endl;  
        break;  
    case 2:
```

```

        cout << "Selasa" << endl;

        break;

    case 3:

        cout << "Rabu" << endl;

        break;

    default:

        cout << "Hari lain" << endl;

}

```

- for : Melakukan sesuatu berulang kali sebanyak yang ditentukan.

cpp

```

for (int i = 0; i < 5; i++) {

    cout << i << " "; // Menampilkan angka 0 sampai 4

}

cout << endl;

```

- while : Melakukan sesuatu berulang kali selama kondisi benar.

cpp

```

int angka = 1;

while (angka <= 3) {

    cout << angka << " ";

    angka++;

}

cout << endl;

```

- do-while : Sama seperti while , tapi dilakukan minimal sekali.

cpp

```

int pilihan;

do {

    cout << "Pilih 1, 2, atau 3: ";

```

```
    cin >> pilihan;  
} while (pilihan < 1 || pilihan > 3);
```

6. Fungsi

- Definisi: Sekumpulan kode yang melakukan tugas tertentu.

cpp

```
int tambah(int a, int b) {  
    return a + b;  
}
```

- Pemanggilan: Menggunakan fungsi.

cpp

```
int hasil = tambah(5, 2); // Memanggil fungsi "tambah" dengan angka 5 dan 2  
cout << "Hasil: " << hasil << endl; // Menampilkan "Hasil: 7"
```

7. Pointer

- Definisi: Variabel yang menyimpan alamat memori variabel lain. (Agak rumit, tapi penting).

cpp

```
int umur = 25;  
  
int *p_umur = &umur; // p_umur menyimpan alamat memori dari variabel umur
```

DASAR TEORI STACK

Pengertian Stack

Stack adalah struktur data linear yang bekerja dengan prinsip **LIFO (Last In, First Out)**, yaitu elemen **terakhir masuk** menjadi elemen **pertama keluar**. Stack hanya memiliki satu titik akses yaitu **TOP**, tempat elemen ditambahkan atau dihapus.

Operasi utama pada stack meliputi:

1. **Push** – menambahkan elemen ke bagian atas stack.
2. **Pop** – mengambil dan menghapus elemen paling atas.
3. **isEmpty** – mengecek apakah stack kosong.
4. **isFull** – mengecek apakah stack penuh (khusus representasi array).
5. **Peek/Top** – melihat elemen teratas tanpa menghapusnya.

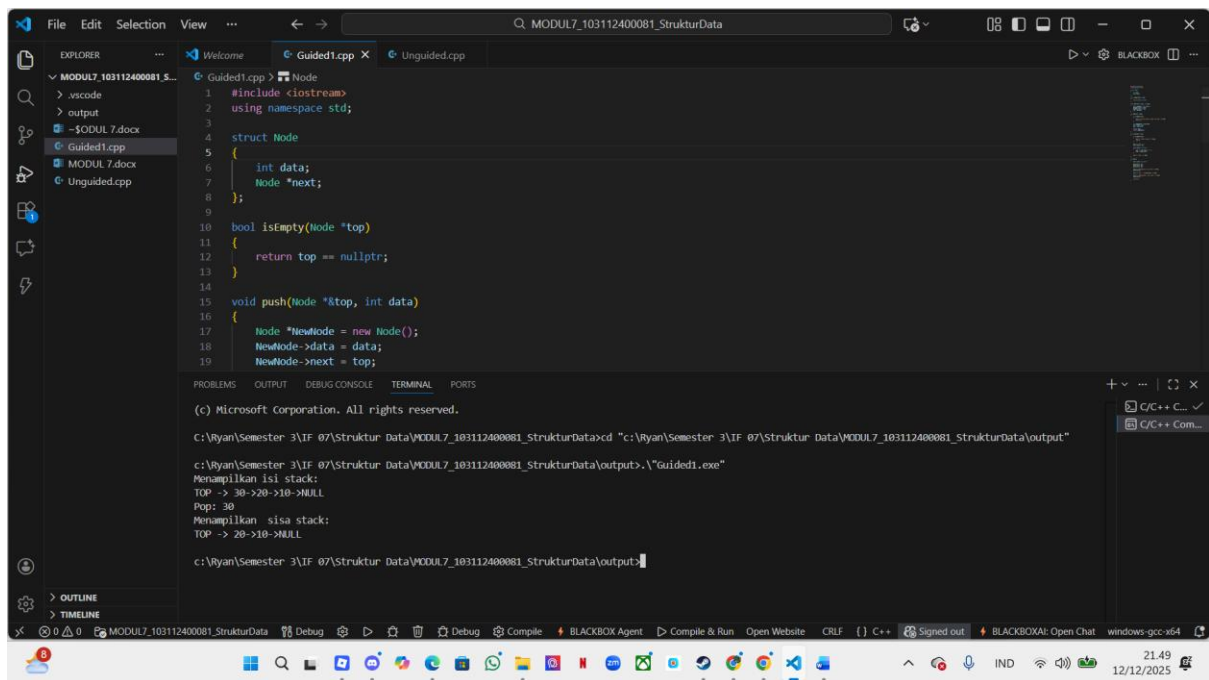
Dalam C++, stack dapat diimplementasikan dengan:

- **Array**, yang sederhana namun ukurannya tetap.
- **Linked List (pointer)**, yang lebih fleksibel karena ukurannya dinamis.
- **STL <stack>**, yaitu implementasi bawaan C++ yang siap pakai.

Stack banyak digunakan dalam kehidupan nyata, seperti proses undo/redo, manajemen pemanggilan fungsi (call stack), evaluasi ekspresi matematika, dan navigasi browser.

Secara keseluruhan, stack merupakan struktur data yang penting dan efisien untuk proses yang membutuhkan pengelolaan data secara berurutan dengan pola LIFO.

Guided 1



```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

bool isEmpty(Node *top)
{
    return top == nullptr;
}

void push(Node *top, int data)
{
    Node *NewNode = new Node();
    NewNode->data = data;
    NewNode->next = top;
}
```

(c) Microsoft Corporation. All rights reserved.

C:\Ryan\Semester 3\IF 07\Struktur Data\MODUL7_103112400081_StrukturData\output

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL7_103112400081_StrukturData\output>.\"Guided1.exe"

Menampilkan isi stack:
TOP -> 30->20->10->NULL
Pop: 30
Menampilkan sisa stack:
TOP -> 20->10->NULL

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL7_103112400081_StrukturData\output>

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita **membuat dan mengimplementasikan struktur data Stack menggunakan Linked List (pointer) dalam bahasa C++**, yang memungkinkan pengguna untuk melakukan operasi dasar stack

Unguided

Coding:

```
#include <iostream>
using namespace std;

const int MAX = 20;

struct Stack {
    int info[MAX];
    int top;
};

void createStack(Stack &S) {
    S.top = -1;
}

bool isEmpty(Stack S) {
    return S.top == -1;
}

bool isFull(Stack S) {
    return S.top == MAX - 1;
}
```

```

}

void push(Stack &S, int x) {
    if (isFull(S)) {
        cout << "Stack penuh, tidak bisa push!" << endl;
        return;
    }
    S.top++;
    S.info[S.top] = x;
}

int pop(Stack &S) {
    if (isEmpty(S)) {
        cout << "Stack kosong, tidak bisa pop!" << endl;
        return -1;
    }
    int x = S.info[S.top];
    S.top--;
    return x;
}

void printInfo(Stack S) {
    if (isEmpty(S)) {
        cout << "Stack kosong." << endl;
        return;
    }
    cout << "TOP -> ";
    for (int i = S.top; i >= 0; i--) {
        cout << S.info[i] << " ";
    }
    cout << endl;
}

void balikStack(Stack &S) {
    Stack temp;
    createStack(temp);

    while (!isEmpty(S)) {
        push(temp, pop(S));
    }

    S = temp;
}

void pushAscending(Stack &S, int x) {
    if (isFull(S)) {
        cout << "Stack penuh!" << endl;
        return;
    }
}

```



```

Stack temp;
createStack(temp);

while (!isEmpty(S) && S.info[S.top] < x) {
    push(temp, pop(S));
}

push(S, x);

while (!isEmpty(temp)) {
    push(S, pop(temp));
}
}

void getInputStream(Stack &S) {
    cout << "Masukkan karakter (ENTER untuk selesai): ";

    createStack(S);
    char c;

    cin.ignore();

    while (true) {
        c = cin.get();
        if (c == '\n') break;
        push(S, c);
    }
}

int main() {
    cout << "Hello world!" << endl;

    Stack S;

    createStack(S);
    push(S, 3);
    push(S, 4);
    push(S, 8);
    pop(S);
    push(S, 2);
    push(S, 3);
    pop(S);
    push(S, 9);

    printInfo(S);

    cout << "balik stack:" << endl;
    balikStack(S);
    printInfo(S);
}

```

```

cout << "\n=== PUSH ASCENDING ===" << endl;
createStack(S);

pushAscending(S, 3);
pushAscending(S, 4);
pushAscending(S, 8);
pushAscending(S, 2);
pushAscending(S, 3);
pushAscending(S, 9);

printlnInfo(S);

cout << "balik stack:" << endl;
balikStack(S);
printlnInfo(S);

cout << "\n=== GET INPUT STREAM ===" << endl;
createStack(S);
getInputStream(S);

printlnInfo(S);

cout << "balik stack:" << endl;
balikStack(S);
printlnInfo(S);

return 0;
}

```

Hasil Coding

```

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL7_103112400081_StrukturData\output>.\"Unguided.exe"
Hello world!
TOP -> 9 2 4 3
balik stack:
TOP -> 3 4 2 9

=== PUSH ASCENDING ===
TOP -> 2 3 3 4 8 9
balik stack:
TOP -> 9 8 4 3 3 2

=== GET INPUT STREAM ===
Masukkan karakter (ENTER untuk selesai): Hello World
TOP -> 100 108 114 111 87 32 111 108 108 101
balik stack:
TOP -> 101 108 108 111 32 87 111 114 108 100

c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL7_103112400081_StrukturData\output>

```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita mengimplementasikan struktur data Stack menggunakan representasi array dalam bahasa C++, serta menguji berbagai operasi dasar dan lanjutan pada Stack.